

Exercises on Polynomial Time and Graph Problems

Question–Answer Notes for Exercises 34.1-1 to 34.1-6

Background

Main Theme

These exercises connect optimization problems, decision problems, encodings, and closure properties of the class **P**. The recurring idea is that polynomial time is measured as a function of the **input length**, not necessarily the numeric value of numbers appearing in the input.

1 Exercise 34.1-1

Question 34.1-1

Define the optimization problem LONGEST-PATH-LENGTH as the relation that associates each instance of an undirected graph and two vertices with the number of edges in the longest simple path between the two vertices. Define the decision problem LONGEST-PATH as

$$\{\langle G, u, v, k \rangle \mid G = (V, E) \text{ is undirected, } u, v \in V, k \geq 0, \\ \text{and there is a simple path from } u \text{ to } v \text{ with at least } k \text{ edges}\}.$$

Show that LONGEST-PATH-LENGTH can be solved in polynomial time if and only if LONGEST-PATH is in **P**.

Answer

The optimization version asks for the exact length of the longest simple path. The decision version asks whether this length is at least a given threshold k .

Optimization Implies Decision

Suppose LONGEST-PATH-LENGTH can be solved in polynomial time. Given an instance

$$\langle G, u, v, k \rangle$$

of LONGEST-PATH, compute the longest path length ℓ from u to v . Accept if and only if

$$\ell \geq k.$$

This adds only a comparison after the polynomial-time optimization algorithm, so LONGEST-PATH is in **P**.

Decision Implies Optimization

Now suppose LONGEST-PATH is in **P**. The length of a longest simple path is between 0 and $|V| - 1$, and also at most $|E|$. Therefore we can find the exact length by binary search over the range

$$0, 1, \dots, |E|.$$

At each step, query the decision algorithm with the current midpoint k :

- if the answer is YES, the longest path length is at least k ;
- if the answer is NO, the longest path length is less than k .

The binary search uses

$$\mathcal{O}(\log |E|)$$

calls to the decision algorithm. Each call has input size polynomially related to the original instance, since $k \leq |E|$ and can be encoded in $\mathcal{O}(\log |E|)$ bits. Therefore the optimization problem can be solved in polynomial time.

2 Exercise 34.1-2

Question 34.1-2

Give a formal definition for the problem of finding the longest simple cycle in an undirected graph. Give a related decision problem. Give the language corresponding to the decision problem.

Answer

The optimization problem LONGEST-SIMPLE-CYCLE maps an undirected graph $G = (V, E)$ to a simple cycle in G with maximum possible number of edges.

Decision Problem and Language

The related decision problem asks:

Given an undirected graph G and an integer $k \geq 0$, does G contain a simple cycle with at least k edges?

The corresponding language is

$$L_{\text{LONGEST-CYCLE}} = \{\langle G, k \rangle \mid G = (V, E) \text{ is an undirected graph, } k \geq 0, \text{ and } G \text{ has a simple cycle with at least } k \text{ edges}\}.$$

3 Exercise 34.1-3

Question 34.1-3

Give a formal encoding of directed graphs as binary strings using an adjacency-matrix representation. Do the same using an adjacency-list representation. Argue that the two representations are polynomially related.

Answer

Assume the vertices are numbered

$$0, 1, \dots, n-1.$$

We give two encodings and then show that each can be converted into the other in polynomial time.

Adjacency-Matrix Encoding

First encode n using a standard self-delimiting binary integer code. Then write n^2 bits in row-major order. The bit in position corresponding to ordered pair (i, j) is

$$\begin{cases} 1, & \text{if } (i, j) \in E, \\ 0, & \text{otherwise.} \end{cases}$$

Thus the matrix encoding has length

$$\Theta(n^2)$$

plus the bits needed to encode n .

Adjacency-List Encoding

Use a delimiter-safe integer code $g(t)$ for vertex indices. One simple choice is to take the usual binary representation of t and insert a 0 after every bit. This doubles the length but ensures that the codeword never contains the substring 11.

For a vertex whose outgoing neighbors are

$$i_1, i_2, \dots, i_k,$$

encode its list as

$$g(i_1)11g(i_2)11 \cdots 11g(i_k)1111.$$

The delimiter 11 separates entries, and the delimiter 1111 ends the current vertex's list. Since no $g(i)$ contains 11, the parsing is unambiguous by counting runs of consecutive 1s. Empty adjacency lists are encoded by 1111.

The full graph encoding is the encoding of n , followed by the n encoded adjacency lists.

Polynomial Relationship

To convert an adjacency list to an adjacency matrix, initialize an $n \times n$ zero matrix. Then scan each list and set the corresponding entries to 1. This takes polynomial time in the input size and produces n^2 matrix bits.

To convert an adjacency matrix to an adjacency list, scan each row of the matrix. Whenever entry (i, j) is 1, output the encoded index $g(j)$ in the list for vertex i . This also takes polynomial time.

The matrix encoding has size $\Theta(n^2)$, and the list encoding has size polynomial in n and $|E|$. Since each representation can be translated into the other in polynomial time, the encodings are polynomially related.

4 Exercise 34.1-4

Question 34.1-4

Is the dynamic-programming algorithm for the 0-1 knapsack problem from Exercise 16.2-2 a polynomial-time algorithm? Explain your answer.

Answer

No. The dynamic-programming algorithm is **pseudo-polynomial**, not polynomial-time in the length of the input.

Explanation

The usual dynamic-programming algorithm for 0-1-KNAPSACK runs in time

$$\Theta(nW),$$

where n is the number of items and W is the knapsack capacity.

The issue is that W is a numeric value, but the input stores W in binary. If the binary encoding of W has length b , then

$$b = \Theta(\log W),$$

so

$$W = \Theta(2^b).$$

Therefore

$$\Theta(nW) = \Theta(n2^b),$$

which is exponential in the number of bits used to encode W .

Thus the algorithm is polynomial in the numeric value W , but not polynomial in the actual input length.

Common Mistake

Saying that $\Theta(nW)$ is polynomial because it is polynomial in n and W ignores how W is encoded. Complexity is measured in the length of the input, not in the magnitude of a binary-encoded number.

5 Exercise 34.1-5

Question 34.1-5

Show that if an algorithm makes at most a constant number of calls to polynomial-time subroutines and performs an additional amount of work that also takes polynomial time, then it runs in polynomial time. Also show that a polynomial number of calls to polynomial-time subroutines may result in an exponential-time algorithm.

Answer

A constant number of polynomial-time calls preserves polynomial time. A polynomial number of calls can fail to preserve polynomial time if the outputs of the calls grow and become the inputs to later calls.

Constant Number of Calls

Suppose the algorithm makes at most c subroutine calls, where c is a fixed constant. Each subroutine runs in polynomial time in the size of its own input, and the algorithm does only polynomial additional work.

Proof Idea

Before the first subroutine call, the algorithm can construct only a polynomial-size query, because it has done only polynomial work. The first subroutine therefore runs in polynomial time and produces at most polynomially many output bits.

After one call, the total amount of information produced so far is still polynomially bounded. By the same reasoning, the next call also receives polynomial-size input and takes polynomial time.

Repeating this argument for a constant number of calls gives only a constant composition and sum of polynomials. Therefore the whole algorithm runs in polynomial time.

Polynomially Many Calls Can Be Exponential

Let the subroutine S take a binary integer x and output x^2 . Squaring an N -bit integer can be done in polynomial time in N , and the output has about $2N$ bits.

Now consider an algorithm that starts with an N -bit integer x and calls S exactly N times, feeding each output into the next call:

$$x, \quad x^2, \quad x^4, \quad x^8, \quad \dots, \quad x^{2^N}.$$

This is only a linear number of subroutine calls in the original input length N . But the final output has

$$\Theta(2^N N)$$

bits. Merely writing this output takes exponential time in N .

Key Point

Polynomially many calls are safe only when the sizes of all subroutine inputs remain polynomially bounded in the original input length. Without that size control, outputs can grow exponentially across calls.

6 Exercise 34.1-6

Question 34.1-6

Show that the class $\mathbf{P}$, viewed as a set of languages, is closed under union, intersection, concatenation, complement, and Kleene star. That is, if $L_1, L_2 \in \mathbf{P}$, then

$$L_1 \cup L_2 \in \mathbf{P}, \quad L_1 \cap L_2 \in \mathbf{P}, \quad L_1 L_2 \in \mathbf{P}, \quad L_1^* \in \mathbf{P}, \quad L_1^c \in \mathbf{P}.$$

Answer

Let A_1 and A_2 be polynomial-time algorithms deciding L_1 and L_2 . We build polynomial-time algorithms for each language operation.

Union and Intersection

On input x , run both $A_1(x)$ and $A_2(x)$.

- For $L_1 \cup L_2$, accept if at least one algorithm accepts.
- For $L_1 \cap L_2$, accept if both algorithms accept.

Two polynomial-time computations still take polynomial time.

Complement

To decide L_1^c , run $A_1(x)$ and flip the answer:

accept if and only if $A_1(x)$ rejects.

This is polynomial time.

Concatenation

To decide L_1L_2 , let $n = |x|$. Try every split

$$x = yz,$$

including the cases where y or z is empty. There are $n + 1$ possible splits. For each split, run $A_1(y)$ and $A_2(z)$. Accept if some split satisfies

$$y \in L_1 \quad \text{and} \quad z \in L_2.$$

This uses only polynomially many polynomial-time checks, so $L_1L_2 \in \mathbf{P}$.

Kleene Star

To decide L_1^* , use dynamic programming. Let $x = x_1x_2 \cdots x_n$. Define

$$\text{dp}[j] = \begin{cases} \text{true}, & \text{if the prefix } x_1x_2 \cdots x_j \text{ is in } L_1^*, \\ \text{false}, & \text{otherwise.} \end{cases}$$

Set $\text{dp}[0] = \text{true}$. For $j = 1, 2, \dots, n$, set $\text{dp}[j]$ to true if there exists an $i < j$ such that

$$\text{dp}[i] = \text{true} \quad \text{and} \quad x_{i+1}x_{i+2} \cdots x_j \in L_1.$$

The second condition is checked by running A_1 on the substring.

There are $\mathcal{O}(n^2)$ pairs (i, j) , and each call to A_1 is on a string of length at most n . Therefore the algorithm runs in polynomial time. Hence $L_1^* \in \mathbf{P}$.

Final Summary

What to Remember

- Optimization and decision versions are often polynomially equivalent when the objective value lies in a polynomially bounded search range.
- Encodings matter: adjacency matrices and adjacency lists are polynomially related, but numeric values such as W in knapsack can be exponentially larger than their binary encodings.
- A constant number of polynomial-time subroutine calls preserves polynomial time.
- Polynomially many subroutine calls can be dangerous if intermediate outputs grow exponentially.
- $\mathbf{P}$ is closed under union, intersection, concatenation, complement, and Kleene star.